

Pavlov: AST 구조 복원을 위한 Diffusion 기반 Decompilation

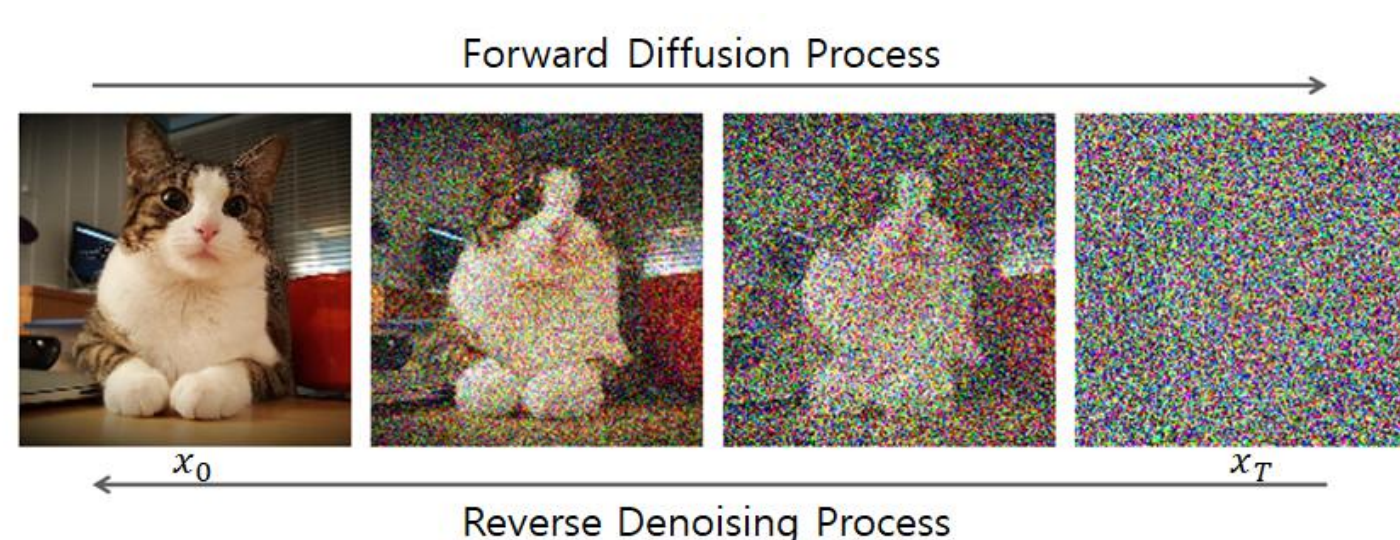
박천웅, 강민석, 권준영, 정우진, 지경선

요약

- 기존 LLM 기반 Decompile은 한 토큰씩 생성하는 Autoregressive 방식에 의존하나, 코드 구조 복원은 전역 제약이 강해 단방향 생성에서 불리함
- Diffusion model은 시퀀스 전체를 양방향으로 정제하므로 전역 구조 복원에 유리할 수 있으며, 본 연구는 AST skeleton 복원에 Diffusion을 적용함
- 동일 계열 기반 모델에서 파생된 Diffusion·AR model을 같은 데이터·같은 크기 LoRA로 학습하여, Skeleton 복원 단계에서 두 방식의 차이를 통제 비교함

배경 지식

- Diffusion Model
 - 데이터에 noise를 조금씩 더해 망가뜨린 뒤, 그 과정을 거꾸로 되돌리도록 학습해서 noise로부터 데이터를 복원하는 생성 모델임
 - 주로 생성형 이미지 모델에 사용 됨



[Fig. 1] Diffusion Model의 정방향 확산 및 역방향 노이즈 제거 과정

- text 기반 diffusion은 image와 달리 token이 이산값이라 gaussian noise 대신 MASK로 치환하는 masked diffusion을 사용함
- Decompile
 - Decompile은 컴파일된 프로그램(바이너리)를 사람이 읽을 수 있는 고수준 소스 코드로 다시 복구하는 과정
 - 디스어셈블 → IR 승격 → 제어흐름·데이터흐름 분석 → 타입 복원·구조화(=AST 복원) → code 생성 순으로 구성됨

제안 방법

- Step 1: 데이터 전처리
 - LLM4Binary/Decompile-bench에서 source 함수와 컴파일된 binary를 가져와, binary는 split zip에서 필요 구간만 부분 추출
 - Ghidra analyzeHeadless로 함수별 pseudocode를 jsonl로 추출
 - named 함수만 project·함수명 키로 source와 매칭하고, tree-sitter로 AST를 순회해 식명화하여 skeleton 생성
- Step 2: 학습
 - r 32, alpha 64, 학습 파라미터 약 80M으로 전체의 1.05%로 설계
 - A arm: autoregressive
 - Qwen2.5-Coder-7B에 표준 cross entropy, prompt·pad는 -100 마스킹
 - D arm: diffusion
 - DiffuCoder-7B-Base에 masked diffusion 적용.
 - LLaDA recipe로 target을 확률 t로 [MASK] 치환 후 masked 위치 cross entropy에 1/t 가중
- DiffuCoder-7B-Base는 Qwen2.5-Coder-7B와 같은 계열 모델로, 두 arm의 backbone은 동일
- Step 3: 평가
 - 평가셋은 Decompile-eval 240 balanced subset = 60 함수 × 00-03 으로 구성
 - Ghidra pseudocode에서 각 arm으로 skeleton 생성
 - frozen model filler가 placeholder를 채워 컴파일 가능한 C 복원
 - 생성 skeleton과 정답 skeleton의 AST를 node type 기준 edit distance로 비교해 구조 복원 정확도 측정
 - 구조 보존은 filler 출력 재식명화 후 입력 skeleton과 비교한 skeleton 위반율로 검증
- filler를 두 arm 공통으로 사용하므로 최종 성능 차이는 두 arm의 학습 능력 차이

실험 및 결과

- 실험 설정
 - 환경: A100 80GB, torch 2.9.1+cu130, transformers 4.51.3, peft 0.19.1
 - 모델: D: DiffuCoder-7B-Base, A:Qwen2.5-Coder-7B
 - LoRA r32 / α 64, bf16, 동일 학습량
 - 학습: train_set 17,760, 2000 step, effective batch 16, gen-len 512, seed 0/1
 - 비교 방법: A·D arm + 공통 frozen filler
 - 생성: A는 greedy, D는 diffusion_generate
 - 이때, D는 denoising step을 충분히 적용
 - 지표
 - AST edit distance: 생성 skeleton과 정답 skeleton AST의 node type 기준 tree edit distance
 - skeleton 위반율: filler 출력이 입력 skeleton과 구조가 달라진 비율
 - re-executability: 복원한 C를 컴파일·실행해 원본 test를 통과하는 비율

실험 결과 분석

arm	AST edit distance	skeleton 위반율	re-executability
A arm (Autoregressive)	0.534	82.9%	26.7%
D arm (Diffusion)	0.531	75.8%	20.4%

[table. 1] seed 0 평가 결과

arm	AST edit distance	skeleton 위반율	re-executability
A arm (Autoregressive)	0.504	83.8%	27.5%
D arm (Diffusion)	0.484	76.2%	22.1%

[table. 2] seed 1 평가 결과

- 핵심지표 : AST edit distance
 - seed0, seed1 모두 사실상 대등하거나 우위. AST edit distance에서 diffusion이 AR baseline보다 높은 구조 복원력을 보임
 - 함수 길이별로 나누면 짧은 함수는 A가 우위, 함수가 길어질수록 격차가 D쪽으로 좁혀짐
 - 전역 구조가 많아질수록 양방향 생성의 이점이 드러나는 경향
- Re-executability, skeleton 위반율
 - Re-exec는 A가 D보다 높음. 다만 두 지표 모두 frozen filler가 pseudocode로 재 생성하는 영향을 크게 받음
 - D의 skeleton 위반율은 두 실험에서 모두 낮음
 - filler가 채우는 과정에서 D의 skeleton을 덜 수정했음을 의미

결론 및 고찰

- 본 연구는 skeleton 복원 단계에 diffusion을 적용해 autoregressive와 동일 조건에서 통제 비교함
- AST edit distance에서 diffusion은 AR baseline과 대등하거나 이상이고, skeleton 위반율도 더 낮아, 두 구조 지표가 함께 diffusion의 구조 복원이 유의미한 수준이었음을 뒷받침함
- 확률적 변동성을 점검하고자 seed를 달리해 두 번 반복함. 두 seed 모두 실험결과가 일관되게 관측 됨
- 제약된 컴퓨팅 자원 탓에 정작 그 이점이 가장 잘 드러날 긴 함수와 multi-function long-context regime을 충분히 다루지 못함. 자원을 확보해 이 regime을 본격적으로 평가하는 것이 확장 방향임

